Now Node B is the only node in the cluster, and it has three shards. As the replicas no longer exist, the cluster's status is set to YELLOW. Once DevOps brings up Node A, the shards will be rebalanced and reassigned, and replicas will be instantiated, so the system will try to attain a healthy GREEN state. Note that Elasticsearch manages such disasters and can exist with minimal resources if needed, to avoid downtime, and it does so behind the scenes without us having to worry about any of those operational headaches.

SHARD SIZING

A common question is how to size shards. There is no one-size-fits-all answer. To get a conclusive result, we must perform sizing trials (with due diligence) based on the organization's current data requirements and future needs. Industry best practice is to size an individual shard as no more than 50 GB, but I have seen shards up to 400 GB in size. GitHub's indexes are spread across 128 shards, each 120 GB. I advise making shards between 25 GB and 40 GB, keeping the node's heap memory in mind. If we know that a `movies` index may hold up to 500 GB of data, it is best to distribute this data among 10 to 20 shards distributed across multiple nodes.

There is one more parameter to consider when sizing shards: heap memory. As we know, nodes have limited computing resources, such as memory and disk space. Each Elasticsearch instance can be tweaked to use heap memory based on available memory. I advise hosting up to 20 shards per gigabyte of heap memory. By default, Elasticsearch is instantiated with 1 GB memory, but the setting can be changed by editing the jvm.options file in the config directory of the installation. Tweak the `Xms` and `Xmx` properties of the JVM to set the heap memory based on your availability and needs.

The takeaway is that shards hold our data, so we must do the initial legwork to get the sizing right. Sizing depends on how much data the index holds (including future requirements) and how much heap memory we can allocate to a node. Every organization must have a strategy for shards before onboarding data. It is imperative to balance data requirements and the optimal number of shards.

Shards cannot be modified on a live index

The number of shards cannot be changed once the index is created and is in operation. When we create an index, Elasticsearch associates a single shard with a single replica by default (before version 7, the default was five shards and one replica). While the number of shards is written in stone, the number of replicas can be changed using the index settings API during the course of the index's life. Documents are housed in a particular shard according to the routing algorithm:

```
shard_number = hash(document_id) % number_of_primary_shards
```

The algorithm depends directly on the number of shards, so changing that number during a live run will modify the current document's location and corrupt it. This in turn will compromise the inverted index and retrieval process. There's a way out, though: reindexing. By reindexing, we can change the shard settings if necessary. We learn about the reindexing mechanism in detail in chapter 5.